

퍼블리싱의 견고함에 데이터 로직을 더해,
사용자 경험을 완성하는
김민중입니다.

2026 PORTFOLIO

프론트엔드 직무 지원자

퍼블리셔로 3년 이상 근무하며 다양한 UI를 구현한 경험을 바탕으로,
프론트엔드 영역으로 확장해 실제 서비스 로직까지 다루고 있습니다.
프론트엔드 개발자로서 UI 구현을 넘어 데이터 흐름과 구조 설계를 함께
고민하며 사용자 경험을 개선하는 데 집중하고 있습니다.



2000년생 (26세)

휴대폰: 010-3412-2137

이메일: f1minn2r@naver.com

주소: (15402) 경기도 안산시 단원구

경력

(주) 아이스크림에듀

총 3년 8개월

학력

신길고등학교

고등학교 졸업

취미 활동

3d blender

사이트

GitHub

업무 경험 3년 8개월

주식회사 아이스크림에듀 | 전략기획팀 | 사원 2023년 9월~2025년 10월 (2년 2개월)

다양한 서비스의 UI 구현과 구조 개선에 참여하며 데이터 처리와 사용자 흐름을 고려한 인터페이스를 개발했습니다.
서로 다른 데이터 구조를 하나의 렌더링 로직으로 통합해 공통 UI를 재사용할 수 있도록 설계하며 유지보수 효율을 높였습니다.
또한 레거시 코드 리팩토링과 공통 스크립트 통합을 통해 코드 중복을 줄이고 서비스 구조를 개선한 경험이 있습니다.

주식회사 위즈클라쓰 | 개발파트 | 사원 2023년 6월~9월 (4개월)

신규 프로젝트의 초기 설계 단계에 참여하고 주요 페이지 퍼블리싱을 주도하며 서비스 완성도를 높였습니다.
React와 Vue 기반 프로젝트의 유지보수 및 리팩토링을 통해 코드 가독성을 개선하고 구조를 정리했습니다.
또한 디자이너와 협업하며 UI/UX 품질을 높인 경험이 있습니다.

주식회사 파이프센스 | 개발파트 | 사원 2022년 2월~2023년 4월 (1년 3개월)

내부 서버 이전 프로젝트에 참여해 레거시 시스템을 자사 환경으로 이전하고 PHP 기반 구조를 최적화하며 서비스 안정성을 확보했습니다.
고객사 프로젝트 퍼블리싱과 검수 및 CS 대응을 수행하며 UI/UX를 개선하고 기술 지원 체계를 정리해 사용자 및 고객사 만족도를 높였습니다.
또한 신규 기능 개발과 유지보수 프로세스 개선을 통해 서비스 품질을 향상시키고 대응 속도를 단축한 경험이 있습니다.

학력 고등학교 졸업

신길고등학교

2015년 3월~2018년 2월

교육 과정

프론트엔드 과정 | kt cloud 2025년 9월~2026년 4월

- 사이버보안, AI, 기획, 디자인 등 7개 직군이 참여하는 통합 프로젝트에서 전반적인 프론트엔드 개발 담당
- 컴포넌트 기반 설계 및 TypeScript를 활용한 코드 안정성 확보, 렌더링 최적화 역량 습득

반응형UIUX웹퍼블리셔디자인 | 라인컴퓨터아트학원 2021년 9월~2022년 2월

- 디자인 시안을 바탕으로 UI를 구현하며, 다양한 디바이스 환경에서 일관된 사용자 경험을 제공하는 레이아웃 구현

프로젝트

POP-CON

kt cloud 실무 통합 프로젝트

프로젝트 소개

프론트엔드

URL: <https://popcon.store/>

백엔드

URL: <https://api.popcon.store/>

팀명	T1
프로젝트명	팝콘(POP-CON)
	인기 팝업스토어의 입장 예약을 공정하고 안정적으로 처리하면서, 초과 수요를 합법적이고 투명한 방식으로 수익화하는 Two-Track(추첨/경매) 기반 팝업 전용 플랫폼
팀원 구성	- 기획/디자인 (4명) - 개발 (5명 - 프론트(2)/백엔드(3)) - 생성형 AI (3명) - 클라우드/보안 (5명)
맡은 역할	- 프론트엔드 개발 전반 담당 (페이지 구현 및 기능 개발) - API 연동 및 상태 관리 구조 설계 - 공통 컴포넌트 설계 및 재사용성 개선 - 프론트엔드 작업 문서화 및 협업 프로세스 관리 - GitHub 레포지토리 및 이슈 관리
기간	2026. 01. 14. ~ 2026. 04. 20.

사용 기술 스택

구분	기술명	선정 이유 및 용도	비고
Framework	Next.js	SEO 최적화 및 초기 로딩 속도 개선 (SSR)	App Router 사용
Language	TypeScript	정적 타입 지정으로 런타임 에러 방지, 팀 협업 시 코드 안정성 및 생산성 향상	
State Mgmt	Zustand	액세스 토큰 인메모리 저장 (auth-store) 및 대기열 진입 시 drawId 공유 (queue-store)	
	TanStack Query	대기열 상태 폴링 (refetchInterval: 3000ms) — WAITING → ACTIVE 전환 감지 및 서버 상태 캐싱	

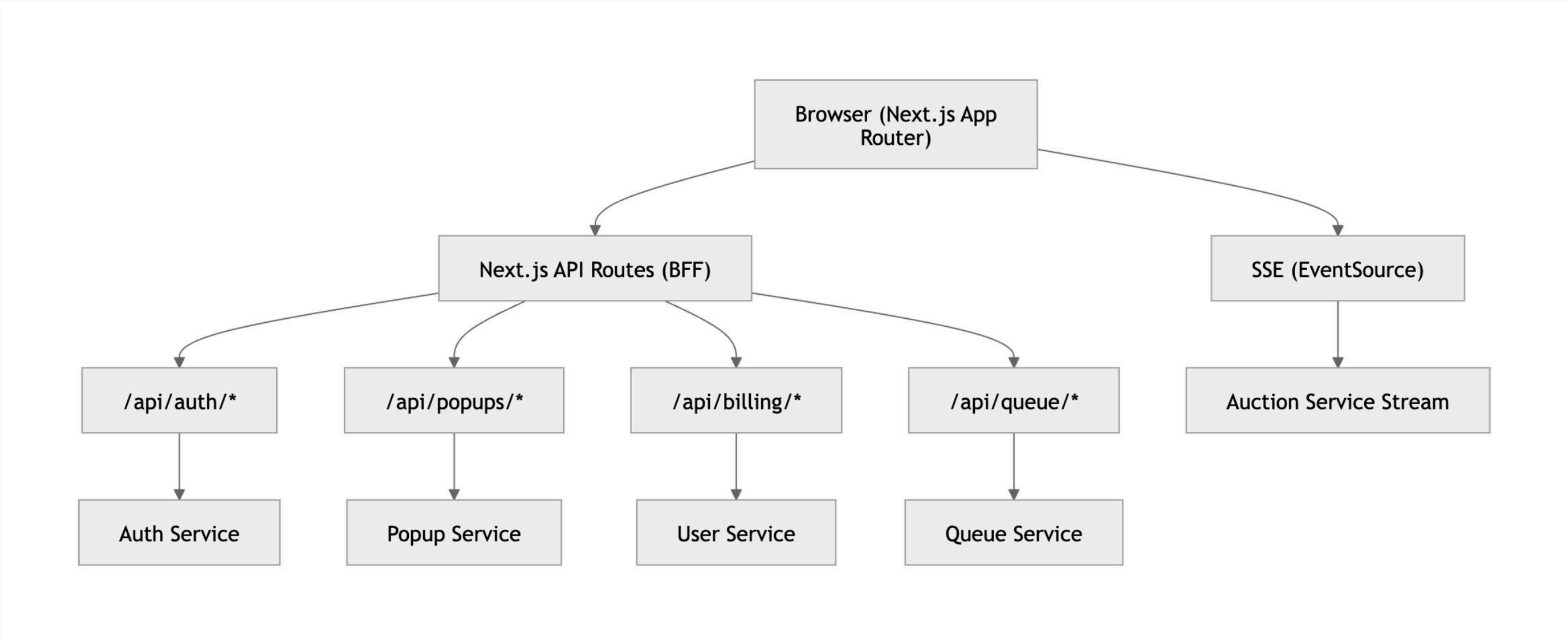
Real-time	Polling (TanStack Query)	대기열 상태 주기적 조회 (3초 간격)	refetchInterval 활용
Styling	Tailwind CSS	빠른 UI 개발 및 모바일 반응형 대응	
UI Components	shadcn/ui	재사용 가능한 컴포넌트 라이브러리, 커스터마이징 용이, radix-ui 기반 접근성	Tailwind 기반
Test Framework	jest	API 라우트 핸들러 (/api/popups/*, /api/auth/*) 및 React 컴포넌트 유닛 테스트	
Code Quality	ESLint + Prettier	코드 스타일 통일 및 품질 관리, 팀 협업 효율성 향상	
Test (Mocking)	MSW	테스트 환경에서 경매 스트림, 대기열, 인증 API 핸들러 모킹 (mocks/handlers/)	
Component Dev	Storybook	Button, Modal, Avatar, Progress 등 공용 UI 컴포넌트 독립 개발/시각적 검증	
Payment	PortOne (@portone/browser-sdk)	회원가입 본인인증	

GitHub

pop-con-frontend
URL: <https://github.com/kt-cloud-TECHUP-T1/pop-con-frontend>

아키텍처

전체 시스템 구조도



폴더 구조 설명

Code BlockBASH

```
1  src/
2  └─ app/           # Next.js App Router (페이지 + API Routes)
3  └─ features/      # 도메인 단위 모듈 (auth, queue, reserve, anti-macro)
4  │                 # → 여러 페이지에서 재사용되는 비즈니스 로직
5  └─ components/    # 도메인 무관 공통 UI 컴포넌트
6  └─ types/         # 전역 공유 타입 (ApiResponse 등)
7  └─ lib/           # 유틸리티 함수
```

데이터 흐름

Code BlockBASH

```
1  컴포넌트 → fetch('/api/popups') → Next.js API Route → Popup 서비스 → DB
2                                     (accessToken 헤더 주입)
```

실시간 경매 흐름

Code BlockBASH

```
1  컴포넌트 → EventSource → Auction 서비스 SSE 스트림 → 가격 이벤트 수신
```

대기열 흐름

Code BlockBASH

```
1  컴포넌트 → TanStack Query (3초 폴링) → Queue 서비스 → ACTIVE 전환 시 퀴즈 페이지 이동
```

작업 구현

페이지 이동으로 상세 작업 내역을 확인하실 수 있습니다.

(1) 공통 컴포넌트

[Notion 보러가기](#)

컴포넌트명	역할	목적	주요 기능
Box	의미 없는 레이아웃 컨테이너	radius, shadow, border, background, padding을 디자인 토큰으로 제어	- as prop 기반 렌더링 - 디자인 토큰 → Tailwind 클래스 변환 - 토큰 외 임의값 폴백
Icon	SVG 아이콘 단일 진입점 컴포넌트	- 아이콘 파일을 직접 import하지 않고 name prop 하나로 렌더링 - TypeScript 자동완성 지원	- name 기반 동적 SVG 렌더링 - iconName 타입 자동완성 - icons·ui-icons 두 카테고리 통합
Avatar	사용자 프로필 이미지 컴포넌트	- 이미지 있으면 사진, 없으면 기본 아이콘으로 자동 폴백 - 편집 가능 여부에 따라 Avatar / AvatarEditable로 분리	- 이미지 없을 때 Icon 폴백 - 디자인 토큰 기반 size 제어 - AvatarEditable로 편집 버튼 확장
PaymentAvatar	결제 수단(카드사) 브랜드 아이콘 아바타	- API 응답의 cardName(한글) 값을 받아 해당 카드사 아이콘을 렌더링 - 매핑되지 않는 카드는 기본 아이콘으로 폴백	- cardName 기반 브랜드 아이콘 자동 매핑 - 미지원 카드 폴백 처리 - 디자인 토큰 기반 size 제어

(2) 페이지

[Notion 보러가기](#)

페이지	주요 기능	기여 내용	기술 포인트
포트원 본인 인증	포트원 SDK 팝업 호출 → 본인인증 완료 API 연동 → 신규/기존 회원 분기 처리	- 포트원 SDK v2 연동 - 에러 코드별 스낵바 메시지 매핑 - 만 14세 미만 모달 처리	- 포트원 팝업 응답의 3단계 분기 처리 (취소/실패/성공) - 에러 코드 → 메시지 상수 맵 패턴 - register_token 쿠키 기반 회원가입 세션 검증
회원가입	약관 동의 체크 → 회원가입 API 연동 → 가입 완료 후 메인 이동	- TERMS 상수 기반 동적 약관 목록 렌더링 - 에러 코드별 스낵바 매핑	- TERMS 상수 → apiKey 로 agreements 객체 동적 생성 - Route Handler에서 register_token 쿠키 + registerToken body 분리 (클라이언트 노출 차단)
메인 페이지	배너·추천·랭킹·경매·드로우·매거진·주목·종료임박 8개 섹션 독립 렌더링	- 섹션별 API 연동 및 카드 UI 구현 - 스켈레톤 로딩 - 로그인 상태 연동	- 섹션마다 독립 fetch + AbortController 패턴 - null(로딩) / [](빈값) / data(성공) - 3단계 상태 구분으로 스켈레톤/빈값/콘텐츠 분기
일정 및 회차 선택	예약 가능 날짜 캘린더 표시 → 날짜 선택 시 회차 목록 조회 → 회차 선택	- 커스텀 캘린더 구현 - 날짜→회차 연계 fetch - 경매/드로우 공용 UI 컴포넌트화	- useReserveDateSlots 커스텀 훅으로 날짜·회차 2단계 fetch 캡슐화 - renderSlot render prop으로 경매/드로우 카드 UI 분기 - 캘린더 currentMonth를 API 응답 전/후/유저 조작 3단계 우선순위로 파생

(3) API

[Notion 보러가기](#)

API 기능	엔드 포인트	역할	설계 포인트
본인인증 완료	/auth/identity/complete	포트원 인증 결과 처리 및 신규/기존 회원 분기	- BFF 프록시 구조 - 민감 쿠키(register_token, refresh_token)를 서버에서만 읽어 백엔드로 전달 - handleProxyResponse로 응답 구조 통일 - 서버 사이드 유효성 검사로 백엔드 불필요 요청 사전 차단
약관 동의 및 가입 완료	/auth/signup	약관 동의 후 가입 처리	
토큰 재발급	/auth/token/refresh	refresh_token으로 accessToken 재발급	

QA 및 버그 리포트

페이지 이동으로 상세 내용을 확인하실 수 있습니다.
[Notion 보러가기](#)

공통 컴포넌트 디렉토리 구조 및 네이밍 통합

기능 요청 배경

- 현재 공통 컴포넌트가 다음 두 경로에 분산되어 있음
- 이로 인해 네이밍 규칙, 파일 구조, 사용 방식이 일관되지 않아 향후 디자인 시스템 확장 및 협업 시 혼란이 발생할 수 있음

→ 따라서 공통 컴포넌트의 위치 및 네이밍 규칙을 통합하고, 일관된 구조로 정리하는 리팩토링 작업 진행 필요

현행 문제점

Code BlockBASH

```
1 components/common
2 components/ui (shadcn 기반)
```

개선 제안 사항

- 디렉토리 전략 결정: 공통 컴포넌트 최종 위치 확정
- 네이밍 규칙 통일:
 - Common 접두사 유지/제거 결정
 - PascalCase 파일명 통일 여부 결정

첨부자료

- 이슈: [#13\[REFACTOR\] 공통 컴포넌트 디렉토리 구조 및 네이밍 통합](#)
- PR: [#19\[Refactor\] 공통컴포넌트 디렉토리분리 및 케밥케이스 적용](#)

개인 회고

아쉬운 점

- 초기에는 기능 단위로 PR을 나누려고 했으나, 하나의 기능 구현 과정에서 수정 범위가 점점 확장되면서 변경사항이 누적되어 방대한 양의 PR을 올리는 경우가 발생
- 이로 인해 리뷰어가 변경 맥락을 파악하기 어려웠고, 코드 리뷰가 원활하게 진행되지 못해 대면 설명으로 이어지는 경우가 생김

회고

- PR은 단순히 기능 단위가 아닌, 리뷰어가 맥락을 빠르게 이해할 수 있는 크기로 나누는 것이 중요하다는 점을 느낌
- 이후에는 기능이 완전히 끝나기 전이라도 Draft PR을 먼저 생성하거나, UI 변경 / 로직 변경 등으로 단위를 나누어 변경 범위를 관리하는 방식으로 개선할 필요성을 느낌